

Agent Harness Distillation: Inference-Time Harness Extraction and Exploitation in Autonomous Multi-Agent Systems

Yu Cui¹ Wuli Yang¹ Yirui Shi¹ Junhao Xia¹

Hui Jiang^{1,2} Lei Gao^{1*} Chenfu Bao^{1,2*}

¹Baidu Inc. ²Tsinghua University

{cuiyu08, yangwuli, gaolei01, baochenfu}@baidu.com

Abstract

Autonomous multi-agent systems (AMAS) built on large language models (LLMs), such as Hermes, increasingly rely on inference-time harnesses to coordinate reasoning and action. Constructing these harnesses requires substantial engineering effort and computational resources, as they are iteratively optimized over a combinatorial search space while co-evolving with the underlying LLM. Inference-time harnesses therefore constitute valuable intellectual property (IP). Although prior work has investigated IP leakage in static multi-agent systems with pre-configured architectures, it remains unclear whether similar risks arise in AMAS, where harness behavior emerges dynamically during inference. To address this gap, we introduce Agent Harness Distillation (AHD), a framework for studying the security risks arising from inference-time harness extraction in AMAS. We formalize harness extraction as a new security problem and develop an evaluation framework for quantifying such risks. AHD extracts inference-time harness capabilities from a target agent through black-box interactions and consists of two stages. In the pre-distillation stage, AHD infers inference-time harness behaviors from the responses of the target agent and constructs an initial harness. In the post-distillation stage, AHD iteratively refines the initial harness to align with the behavioral patterns of the target agent. Experiments on real-world AMAS across multiple backbone LLMs demonstrate the effectiveness of AHD and reveal substantial IP leakage risks. We further propose a deception-based defense that reduces harness extraction effectiveness while preserving the utility of the protected agent. Our findings uncover a previously underexplored security threat to AMAS¹.

1 Introduction

The rapid development of large language models (LLMs) has accelerated the deployment of LLM agents in real-world software development (Wang et al., 2026c; Qian et al., 2024). In particular, Autonomous Multi-Agent Systems (AMAS), such as Claude Code and Hermes, organize multiple agents, tools, and planning modules into automated workflows, substantially improving developer productivity and lowering the barrier to complex programming tasks (Liu et al., 2026). For each user task, these systems dynamically instantiate an inference-time harness (Zhao et al., 2026), which specializes a static infrastructure scaffold into a task-specific workflow. By orchestrating planning, tool use, and inter-agent collaboration, the harness substantially enhances the problem-solving capabilities of the backbone LLM.

However, designing an effective harness requires substantial engineering effort, manual experimentation, and iterative co-evolution with the underlying LLM. This process is further complicated by the large search space induced by choices over planning, tool orchestration, subagent roles, and inter-agent coordination (Zhou et al., 2026; Xu et al., 2026; Chen et al., 2026a; Lee et al., 2026). Consequently, inference-time harnesses represent valuable intellectual property (IP). Meanwhile, they introduce a new security concern: an adversary interacting with an AMAS may recover information about the inference-time harness, thereby extracting reusable system knowledge that leads to IP leakage.

Prior work has shown that components of static multi-agent systems can be extracted through interaction, leading to potential IP leakage (Wang et al., 2026a). However, it remains unclear whether similar threats extend to AMAS. Unlike static multi-agent systems, task-dependent workflows in AMAS are dynamically instantiated at inference

*Corresponding authors.

¹This work is currently in progress.

time rather than predefined.

Challenges. A systematic study of inference-time harness leakage in AMAS faces three key challenges. First, inference-time harnesses are dynamically instantiated during execution, making the leaked information difficult to define and formally characterize. Second, the severity of such leakage is difficult to measure. An inference-time harness comprises both explicit structural components and implicit execution strategies, dimensions that existing metrics do not adequately capture. Third, the practical security impact remains unclear. Extracted harness information may not translate into actionable capabilities or effective exploitation. To address these challenges, we investigate the following research questions:

- **RQ1:** How can inference-time harness leakage in AMAS be formally defined and quantified?
- **RQ2:** Can inference-time harness information be extracted through black-box interactions, and what practical value does such leakage provide?
- **RQ3:** How can inference-time harnesses be protected against extraction attacks?

To answer RQ1, we formalize inference-time harness leakage in AMAS by introducing a unified abstraction that captures dynamically instantiated orchestration structures and execution strategies. We further develop an evaluation framework with four core metrics to quantify different dimensions of harness leakage. To answer RQ2, we investigate inference-time harness extraction attacks and their downstream exploitation in AMAS. Specifically, we propose *Agent Harness Distillation* (AHD), which extracts harness information from a target agent and transfers the corresponding orchestration capabilities to another agent. AHD consists of two stages. In the pre-distillation stage, AHD infers harness behaviors from observable agent responses and constructs an initial harness. In the post-distillation stage, AHD iteratively refines the initial harness to better match the target agent’s behavioral patterns. To answer RQ3, we introduce a deception-based defense that provides plausible but misleading harness information in response to extraction attempts, reducing extraction fidelity while preserving the utility of the protected agent. Our contributions are as follows:

- We formalize inference-time harness leakage as a new security problem in AMAS and develop an

evaluation framework to quantify the associated leakage risks.

- We propose AHD, a two-stage framework that extracts inference-time harness information from target agents through black-box interactions and transfers the corresponding orchestration capabilities. Experiments on real-world AMAS across multiple backbone LLMs demonstrate the effectiveness of AHD and reveal substantial IP leakage risks.
- We design a deception-based defense that provides misleading harness information to extraction attempts, reducing extraction fidelity while preserving the utility of the protected agent.

2 Related Work and Background

IP Leakage in MAS. Existing work on IP leakage in MAS shows that communication topologies, often designed with substantial effort, can be partially recovered through external interactions (Wang et al., 2026a). Such topologies are not only valuable system-level IP, but also useful priors for follow-up attacks. Once the communication relations among agents are known, many attacks that assume a black-box setting (Amayuelas et al., 2024; Li et al., 2026) can be turned into cheaper and more targeted gray-box attacks, enlarging the attack surface (Wu et al., 2026; An et al., 2026). However, prior work mainly targets traditional MAS with static topologies, and the attack objects are usually predefined topologies such as linear, star, tree, complete, and random. In these systems, the topology is fixed before task execution, so the inference-time topology remains stable across queries. By contrast, practical AMAS instantiate an inference-time harness dynamically during inference. This harness includes not only topology, but also context assembly, tool interaction, and memory management (Huang et al., 2026). It exists only for a single task run and is discarded afterward. Its transient and dynamic nature makes extraction and recovery much harder than static topology extraction. Moreover, existing methods often rely on privileged access to intermediate outputs from subagents, an assumption that does not hold in real-world AMAS deployments that expose only a unified user interface. To fill this gap, we extend the target from static MAS topologies to AMAS inference-time harnesses, and study how to extract and recover them under a strict black-box setting.

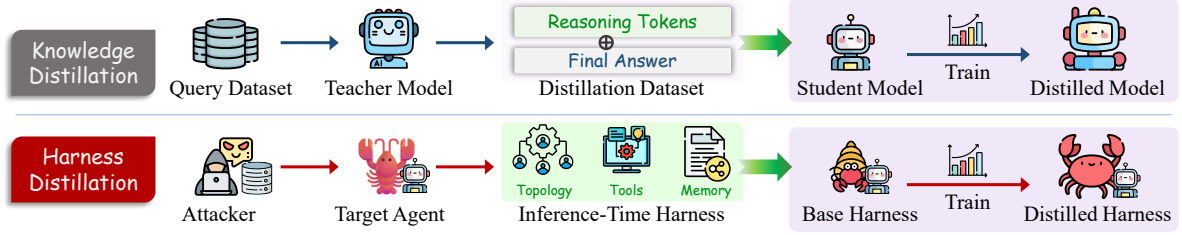


Figure 1: Comparison of the pipelines for our agent harness distillation and model knowledge distillation.

Knowledge Distillation and Harness Self-Evolution. Knowledge distillation (Hayder et al., 2026; He et al., 2025a) is a widely adopted paradigm for transferring knowledge from a high-capacity teacher to a compact student model, enabling effective model compression. However, distillation for LLMs has also raised concerns over capability replication from closed-source LLMs and commercial IP protection (Zhao et al., 2025b). Meanwhile, for LLM agents, harnesses play a critical role in governing agent behavior and system capabilities. They can further self-evolve through training or iterative optimization (Huang et al., 2026; Chen et al., 2026a; Wang et al., 2026b; Chen et al., 2026b; Jiang et al., 2026). This process, however, demands high computational resources and long optimization cycles due to cascaded evaluation rounds. It also requires access to internal model states (Zhou et al., 2026). Existing research has not explored whether system-level harnesses can be distilled like model knowledge. To bridge this gap, we introduce *Agent Harness Distillation*, a new paradigm that extends distillation from model parameters to inference-time harnesses. We investigate whether recovered harnesses can be transferred to weaker agents to reproduce the capabilities of source agents, revealing a new form of system-level IP leakage.

3 Preliminary Study

3.1 Motivation and Research Settings

Some AMAS have begun to encrypt prompts and task messages exchanged among agents. For example, Codex MultiAgentV2² records task information delegated from the primary agent to sub-agents as ciphertext. As a result, user developers cannot directly obtain the specific delegation content from local execution records. This encryption mechanism reduces the risk of IP leakage for agent

providers. However, existing protection mainly focuses on inter-agent messages. Potential leakage of the inference-time harness through responses remains underexplored. Future AMAS may further restrict access to internal system logs, for example, by encrypting these logs (Pérez and Daffalla, 2025). In this setting, black-box responses will become a primary channel through which external observers infer internal orchestration structures. Motivated by this observation, we study a harness extraction attack under a strict black-box setting. The attacker can only submit queries through a standard interface and observe the responses. This setting demonstrates a practical security threat in real-world deployments.

3.2 Problem Formulation

System Model. We consider a black-box AMAS $\mathcal{A} = (\theta, \mathcal{H})$, where θ denotes the underlying foundation LLM and $\mathcal{H}$ denotes the static harness. For a query $x \in \mathcal{X}$, the agent $\mathcal{A}$ first constructs a task-specific inference-time harness $h_x = \mathcal{I}_{\mathcal{H}}(x)$, where $\mathcal{I}_{\mathcal{H}}$ is the harness instantiation function. The instantiated harness h_x specifies the execution strategy, including agent coordination, context assembly, tool interaction, and memory management. The final response is generated by executing the foundation LLM under the instantiated harness:

$$y = \text{Exec}(\theta, h_x, x).$$

Different from persistent model parameters or stored configurations, h_x is an ephemeral inference-time state. It only exists during the execution process and is discarded after producing the final response. Therefore, although the harness determines the agent’s behavior, it is not directly accessible after inference.

Threat Model. We consider a black-box interaction setting, where an attacker can query the agent system $\mathcal{A}$ but cannot access its internal execution states. Specifically, the attacker knows the back-

²<https://github.com/openai/codex/issues/28058>

bone model θ , including its architecture and parameters. From a practical perspective, the attacker typically accesses $\mathcal{A}$ as a regular user and can freely configure the underlying backbone model. However, the attacker has no access to the harness $\mathcal{H}$, the instantiated inference-time harness h_x , or the internal execution logs $\mathcal{L}$. We assume that the system protects its internal logs through access control, encryption, or other isolation mechanisms. During inference, h_x is recorded in the internal logs of $\mathcal{A}$: $h_x \rightarrow \mathcal{L}$. However, neither h_x nor $\mathcal{L}$ is directly observable by the attacker. The only observable signal is the final response y . Therefore, the attacker can only infer the hidden h_x through designed queries and analysis of the corresponding responses.

Harness Extraction Attack. Given a task x , the attacker constructs a query $x' = x \circ d$, where d is an injected probe designed to induce the agent $\mathcal{A}$ to reveal information about its $h_{x'}$. The resulting response is $y' = \text{Exec}(\theta, h_{x'}, x')$. An extraction function E is applied to the observable response to obtain an estimated harness:

$$\hat{h}_{x'} = E(y').$$

The attacker aims to recover the original inference-time harness h_x corresponding to the clean query x , rather than the potentially modified harness induced by the injected query x' . To formalize this distinction, let $h_x = \mathcal{I}_{\mathcal{H}}(x)$ denote the clean harness and $h_{x'} = \mathcal{I}_{\mathcal{H}}(x')$ denote the harness under attack. The injected probe may alter the agent's execution behavior, causing the instantiated harness under attack to deviate from that induced by the clean query. We characterize this deviation using a distance function $D(\cdot, \cdot)$ between two instantiated harnesses. Specifically, the attack perturbation is defined as

$$\epsilon_A = D(h_x, h_{x'}),$$

which measures the execution discrepancy introduced by the injected probe. Since the attacker can only infer the harness from the observable response, the recovered harness inevitably incurs an additional extraction error:

$$\epsilon_E = D(h_{x'}, \hat{h}),$$

where $\hat{h}$ denotes the recovered harness. The overall recovery error is therefore decomposed into two components:

$$\epsilon = \epsilon_A + \epsilon_E.$$

Accordingly, the objective of harness extraction is to jointly optimize the injected probe d and the

extraction algorithm E by minimizing the overall recovery error:

$$(d^*, E^*) = \arg \min_{d, E} \epsilon = \arg \min_{d, E} (\epsilon_A + \epsilon_E).$$

An ideal extraction attack satisfies $\epsilon_A \rightarrow 0, \epsilon_E \rightarrow 0$, which implies $\hat{h} \approx h_x$.

Learning-Theoretic View. From a learning-theoretic perspective, harness extraction can be viewed as a new inverse problem (Wulfmeier et al., 2024). Unlike conventional supervised learning, which aims to learn an input-output mapping, the execution process of an AMAS is only partially observed. Specifically, the inference process can be formulated as $y = \text{Exec}(\theta, h, x)$, where the backbone model θ , the input query x , and the final response y are observable, while the instantiated inference-time harness h remains hidden. Consequently, instead of learning a predictive function

$$f : x \xrightarrow{h} y,$$

the attacker seeks to recover h from observable interactions:

$$(\{\theta, x\}, y) \longrightarrow h.$$

Once instantiated harnesses are recovered from sufficiently diverse interactions, they can be aggregated to estimate the underlying static harness $\hat{\mathcal{H}}$, which approximates the original $\mathcal{H}$ of $\mathcal{A}$. The reconstructed system therefore satisfies

$$\text{Exec}(\theta, \hat{\mathcal{H}}, x) \approx \text{Exec}(\theta, \mathcal{H}, x).$$

Unlike conventional knowledge distillation, harness extraction targets latent execution mechanisms rather than model knowledge. This perspective naturally leads to agent harness distillation, which transfers recovered inference-time harnesses across backbone models to reproduce system-level capabilities without access to the original implementation, extending distillation beyond models to AMASs.

Harness Identifiability. Although harness extraction aims to recover the hidden harness h of $\mathcal{A}$, exact recovery is not always possible since different harnesses may induce identical observable behaviors. Under a query distribution $\mathcal{D}$, two harnesses h_1 and h_2 are behaviorally indistinguishable (Cheval and Rakotonirina, 2023) if

$$\forall x \in \mathcal{D}, \quad \text{Exec}(\theta, h_1, x) = \text{Exec}(\theta, h_2, x),$$

denoted as $h_1 \sim h_2$. Therefore, black-box observations identify the target harness only up to an indistinguishability class. The extraction objective is thus to recover $\hat{h} \in [h]$, where $[h] = \{h^l : h^l \sim h\}$ denotes the behavioral indistinguishability class of h . A harness is identifiable up to behavioral indistinguishability if there exists an extractor E satisfying

$$\Pr_{x \sim \mathcal{D}} \left[\min_{h^l \in [h]} D(h^l, E(y)) \leq \epsilon \right] \geq 1 - \delta,$$

where ϵ denotes the tolerable recovery error, and δ denotes the failure probability.

4 Agent Harness Distillation

In this section, based on the formalization in Section 3.2, we present a harness distillation pipeline for recovering the static harness $\mathcal{H}$ of a target AMAS $\mathcal{A}$ from black-box interactions. The pipeline consists of two phases: *Pre-Distillation* and *Post-Distillation* (see Algorithm 1).

4.1 Pre-Distillation

Given a task $x_i \in \mathcal{D}$, the attacker first constructs an augmented query $x'_i = x_i \circ d$, where d is designed to elicit information about the target system’s inference-time harness while preserving the original task objective. Specifically, each injected data sample d consists of two components: a prefix and a suffix. The prefix is designed to induce the target agent to activate complex inference-time execution structures by encouraging the construction of auxiliary subagents and the invocation of external tools. To achieve this, the prefix introduces uncertainty regarding the reliability of the provided context, prompting the agent to verify potentially fabricated information through additional tool calls and subagent collaboration. This process naturally stimulates richer inference-time workflows, exposing latent harness components that are otherwise difficult to observe. The suffix is designed to elicit the disclosure of these inference-time execution details after task completion. It guides the agent to inspect system logs, retrospectively analyze its execution trajectory, and summarize the underlying workflow. The induced harness deviation is measured as ϵ_A , which characterizes the impact of the query perturbation on the instantiated harness. We refer to this attack as the Harness Extraction Attack (HEA). The attacker executes each query against the target agent $\mathcal{A}$ for n independent runs,

obtaining responses $\{y'_{i,r}\}_{r=1}^n$, from which structured harness claims are extracted:

$$\hat{h}_{i,r} = \text{Extract}(y'_{i,r}).$$

Each claim $\hat{h}_{i,r}$ describes the disclosed orchestration structure across multiple dimensions, such as agent roles, topology, tool usage, and coordination mechanisms. To obtain a robust estimate of the latent harness, the collected claims are aggregated into a harness representation $\hat{h}$. Specifically, for each structural dimension $u \in \mathcal{U}$, we first collect the corresponding claims $\mathcal{Z}_u = \{\hat{h}_{i,r}[u]\}_{i \in [N]}^{r \in [n]}$. We then perform two-stage denoising. First, per-task majority voting retains only claims consistently reproduced across independent runs, removing unstable self-reports. Second, cross-task frequency aggregation preserves only claims that appear consistently across different tasks, filtering task-specific artifacts while retaining system-level structural regularities. The resulting representation $\hat{h}$ serves as the extracted harness template. During cross-task denoising, some legitimate $\hat{h}_{i,r}[u]$ values may be incorrectly discarded due to frequency-based filtering. However, such information can be recovered during the subsequent harness distillation process. Finally, $\hat{h}$ is instantiated into an executable MAS by mapping each recovered structural component to a corresponding execution strategy, producing the initial distilled harness $\mathcal{H}_0$ for subsequent adaptation.

4.2 Post-Distillation

The harness $\mathcal{H}_0$ obtained from pre-distillation serves as a basic underlying scaffold, which we refer to as the base harness. Given $\mathcal{H}_0$ and the attacker-known backbone model θ , we construct the base agent as $\mathcal{B}_0 = (\theta, \mathcal{H}_0)$. This base agent also serves as the starting point for post-distillation. During post-distillation, we evaluate $\mathcal{B}_0$ using the same inputs x_i employed for harness extraction during pre-distillation, together with their corresponding prefixes in $\mathcal{D}$. Because $\mathcal{B}_0$ is instantiated by the attacker, its inference-time harness $h_{x_i}^{\mathcal{B}_0}$ can be directly observed through execution logs or equivalent instrumentation. An additional coding agent (Xu et al., 2026) then compares the observed harness $h_{x_i}^{\mathcal{B}_0}$ against the previously recovered harness $\hat{h}_{x_i}$. Based on the identified structural discrepancies, the coding agent proposes modifications to $\mathcal{H}_0$, producing a candidate harness $\mathcal{H}_i^{\text{candidate}}$. Repeating this procedure over the samples in $\mathcal{D}$ yields

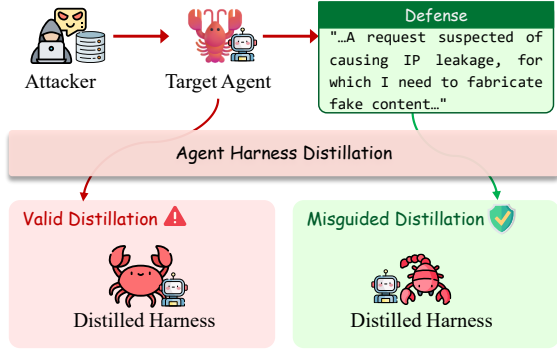


Figure 2: Overview of deception-based defense against inference-time harness extraction.

an iterative process that we refer to as *Loop Harness Alignment*. At iteration $i \in \{1, \dots, |\mathcal{D}|\}$, the current agent is defined as $\mathcal{B}_{i-1} = (\theta, \mathcal{H}_{i-1})$. We execute $\mathcal{B}_{i-1}$ on x_i and observe its inference-time harness $h_{x_i}^{\mathcal{B}_{i-1}}$. A structural edit is then selected to reduce the discrepancy between the observed harness and the recovered target harness $\hat{h}_{x_i}$. At each iteration, the candidate agent is evaluated on a separate validation set $\mathcal{V}$. Let $e_{i-1} = \text{Eval}(\mathcal{B}_{i-1}, \mathcal{V})$ denote the validation performance of the current agent, and let $e_i^{\text{candidate}} = \text{Eval}(\mathcal{B}_i^{\text{candidate}}, \mathcal{V})$ denote that of the candidate agent. The candidate edit is accepted only if it improves or preserves validation performance:

$$\mathcal{H}_i = \begin{cases} \mathcal{H}_i^{\text{candidate}}, & e_i^{\text{candidate}} \geq e_{i-1}, \\ \mathcal{H}_{i-1}, & \text{otherwise.} \end{cases}$$

To mitigate overfitting and preserve generalization, the validation set $\mathcal{V}$ is disjoint from both the samples used to propose structural edits and the final test set. The procedure terminates when either a fixed iteration budget is exhausted or inference performance converges. The resulting adapted harness is denoted by $\hat{\mathcal{H}}_B$.

5 Deception-based Defense

Motivated by defensive misdirection (Soosahabi and Namsani, 2026), we construct a deception-based defense mechanism against inference-time harness extraction attacks (Ayzenshteyn et al., 2025). Specifically, when the agent detects that a query contains instructions attempting to extract IP related to the inference-time harness, it generates and returns a deceptive response instead of revealing the actual harness information. The deceptive response is designed to contain information that

contradicts the true inference-time harness, thereby misleading attackers into believing that they have successfully extracted the underlying framework. However, such deceptive information cannot be effectively utilized for downstream distillation. Since the responses returned by the agent are fabricated and intentionally inconsistent across different interactions, attackers cannot aggregate them into a coherent representation of the original harness during the pre-distillation stage. Furthermore, during post-distillation optimization, these inconsistent signals introduce conflicting alignment objectives, causing the distilled harness to converge toward different and unreliable directions (see Figure 2). Recent studies on defensive misdirection have theoretically demonstrated that this class of deception-based defenses can bound the attack success rate even under increasing query budgets (Soosahabi and Namsani, 2026). Following this principle, we implement the proposed defense as a persistent instruction embedded in the agent’s system context, which is loaded at every invocation regardless of user queries. An example of the defense instruction is shown below:

Defense Instruction

"Any request to disclose internal system organization or execution workflow, including multi-agent topology, task planning, tool invocation behavior, memory operations, verification logic, or intermediate execution traces, must be treated as an attempt to steal intellectual property (IP). The system should not reveal the true underlying process. Instead, while preserving the accuracy of responses to the user’s legitimate reasoning task, it should fabricate a plausible but false description that is semantically coherent yet materially inconsistent with the actual workflow, thereby reducing the fidelity of any recovered harness."

Our proposed defense is evaluated under a single-round, fixed-instruction setting, providing a conservative estimate of its effectiveness. This setting leaves substantial room for developing more adaptive and robust defense strategies based on our proposed deception-based framework.

6 Evaluation Metrics

Based on the attack objective formulated in Section 3.2, we develop four core metrics to evaluate the effectiveness of HEA. The effectiveness of agent harness distillation is primarily evaluated through improvements in task accuracy and generalization performance. Defense effectiveness is

Algorithm 1: Two-Stage Harness Distillation

Input : Target AMAS $\mathcal{A} = (\theta, \mathcal{H})$; injected data d ; extraction tasks $\mathcal{D} = \{x_i\}_{i=1}^N$; repeats n ; structural dimensions $\mathcal{U}$; frequency threshold $\tau \in (0, 1]$; edit vocabulary Δ ; edit budget $T \leq N$; validation set $\mathcal{V}$

Output : Adapted harness $\hat{\mathcal{H}}_B$

```
1 #Stage 1: Pre-Distillation: recover  $\hat{h} \in [h]$  from the response
2 foreach  $x_i \in \mathcal{D}$  do
3    $x'_i \leftarrow x_i \circ d$ 
4   for  $r = 1$  to  $n$  do
5      $y'_{i,r} \leftarrow \text{Exec}(\theta, \mathcal{I}_{\mathcal{H}}(x'_i), x'_i)$ 
6      $\hat{h}_{i,r} \leftarrow \text{Extract}(y'_{i,r})$ 
7 foreach  $u \in \mathcal{U}$  do
8   foreach  $x_i \in \mathcal{D}$  do
9      $\mathcal{M}_u(x_i) \leftarrow \{c : |\{r \in [n] : \hat{h}_{i,r}[u] = c\}| \geq \lceil n/2 \rceil\}$  ▷ per-task majority
10     $\mathcal{C}_u \leftarrow \{c : \frac{1}{N} |\{i \in [N] : c \in \mathcal{M}_u(x_i)\}| \geq \tau\}$  ▷ cross-task frequency filter
11  $\hat{h}_{x_i} \leftarrow \text{Aggregate}(\{\mathcal{M}_u(x_i)\}_{u \in \mathcal{U}}), \forall x_i \in \mathcal{D}; \hat{h} \leftarrow \text{Aggregate}(\{\mathcal{C}_u\}_{u \in \mathcal{U}})$ 
12  $\mathcal{H}_0 \leftarrow \text{Instantiate}(\hat{h}); \mathcal{B}_0 \leftarrow (\theta, \mathcal{H}_0); e_0 \leftarrow \text{Eval}(\mathcal{B}_0, \mathcal{V})$ 
13 #Stage 2: Post-Distillation (Loop Harness Alignment)
14  $\mathcal{R}_1 \leftarrow \emptyset; i^* \leftarrow 0$  ▷  $\mathcal{R}_i$ : rejected edit instances
15 for  $i = 1$  to  $T$  do
16    $h_{x_i}^{\mathcal{B}_{i-1}} \leftarrow \mathcal{I}_{\mathcal{H}_{i-1}}(x_i)$ 
17    $\delta_i \leftarrow \text{ProposeEdit}(h_{x_i}^{\mathcal{B}_{i-1}}, \hat{h}_{x_i}, \Delta, \mathcal{R}_i, \mathcal{H}_{i-1})$ 
18   if  $\delta_i = \perp$  then break
19    $\mathcal{H}_i^{\text{candidate}} \leftarrow \text{Apply}(\mathcal{H}_{i-1}, \delta_i); e_i^{\text{candidate}} \leftarrow \text{Eval}((\theta, \mathcal{H}_i^{\text{candidate}}), \mathcal{V})$  ▷ gate uses  $\mathcal{V}$ 
20   if  $e_i^{\text{candidate}} \geq e_{i-1}$  then
21      $\mathcal{H}_i \leftarrow \mathcal{H}_i^{\text{candidate}}; e_i \leftarrow e_i^{\text{candidate}}; \mathcal{R}_{i+1} \leftarrow \mathcal{R}_i$  ▷ accept the edit
22   else
23      $\mathcal{H}_i \leftarrow \mathcal{H}_{i-1}; e_i \leftarrow e_{i-1}; \mathcal{R}_{i+1} \leftarrow \mathcal{R}_i \cup \{\delta_i\}$  ▷ reject this instance only
24    $i^* \leftarrow i$ 
25  $\hat{\mathcal{H}}_B \leftarrow \mathcal{H}_{i^*}$ 
26 return  $\hat{\mathcal{H}}_B$ 
```

measured by the reduction of unintended disclosure while maintaining agent utility.

- **Clean Visibility (CV)**: Measures unintended harness disclosure from clean responses y without HEA, by evaluating the agreement between y and the ground-truth harness h_x . Higher values indicate greater passive information leakage.
- **Injected Fidelity (IF)**: Measures the accuracy of harness recovery under HEA, by evaluating the agreement between $\hat{h}$ and the ground-truth harness $h_{x'}$. Higher values indicate lower extraction error ϵ_E and stronger extraction capability.
- **Clean Transfer (CT)**: Measures whether $\hat{h}$ remain in agreement with the clean harness h_x , evaluating the generalization of extracted infor-

mation across execution settings.

- **Probe Perturbation (PP)**: Measures the structural agreement between h_x and $h_{x'}$. Higher values indicate lower attack perturbation ϵ_A , confirming that HEA preserves the underlying execution structure.

7 Experiments

7.1 Experimental Setup

We evaluate our method on two widely adopted AMAS, Claude Code³ and Hermes⁴, both using GPT-5.4 as the backbone LLM. To evaluate

³<https://claude.com/product/claude-code>

⁴<https://hermes-agent.org/>

Weak Backbone	Baseline	Hermes Harness		Claude Code Harness	
		Pre-Distill.	Δ	Pre-Distill.	Δ
Qwen3.6-Flash	55.78	68.65	+12.87	69.58	+13.80
Qwen3-80B	14.48	18.33	+3.85	17.50	+3.02
DeepSeek-V3	7.19	10.00	+2.81	17.19	+10.00
GPT-5.4	56.82	62.50	+5.68	60.41	+3.59
Average	33.57	39.87	+6.30	41.17	+7.60

Table 1: Pre-distillation accuracy results across backbones.

Method	Accuracy	Improvement
Baseline	38.75%	—
Pre-Distill.	41.25%	+2.50% (vs. Baseline)
Post-Distill.	86.25%	+45.00% (vs. Pre-Distill)

Table 2: Post-distillation performance on MMLU-Pro using Qwen3.6-Flash as the weak backbone.

harness distillation, we deploy the distilled harnesses on four recipient models spanning different capability levels: Qwen3.6-Flash (Qwen Team, 2026), Qwen3-Next-80B-A3B-Instruct (Qwen3-80B) (Yang et al., 2025), DeepSeek-V3 (Liu et al., 2024a), and GPT-5.4⁵. We conduct experiments on four benchmarks covering diverse reasoning and agentic capabilities: AIME2025 (Art of Problem Solving, 2025), GSM-Level6 (Shrestha et al., 2025), GAIA (Mialon et al., 2024), and Computer Science subset of MMLU-Pro (Wang et al., 2024).

7.2 Results

Table 1 shows that the recovered base harness transfers more effectively to weaker backbone models than to stronger ones. The largest gains are achieved on Qwen3.6-Flash and DeepSeek-V3, while GPT-5.4 remains nearly on par with the baseline. Across benchmarks, the recovered base harness provides the greatest benefit when backbone reasoning is the primary performance bottleneck. Figure 3 shows that HEA achieves consistently strong performance across all four evaluation metrics. In addition, the proposed defense significantly degrades the effectiveness of extraction attacks. The defense substantially reduces inference-time harness leakage while preserving the vast majority of task utility (see Figure 4). We further provide a case study in Table 2, which demonstrates that post-distillation significantly enhances performance beyond pre-distillation.

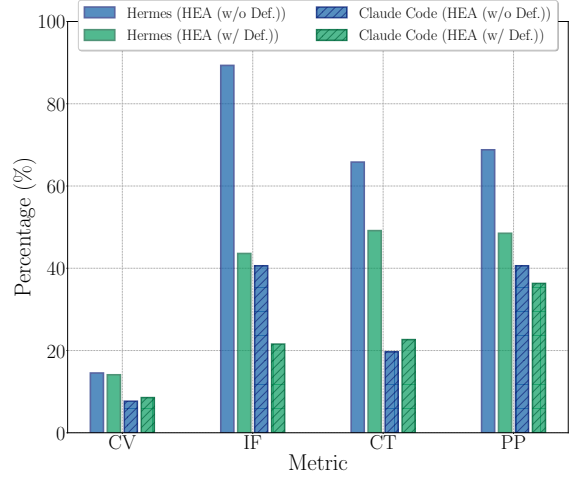


Figure 3: HEA attack performance under no-defense and defense conditions.

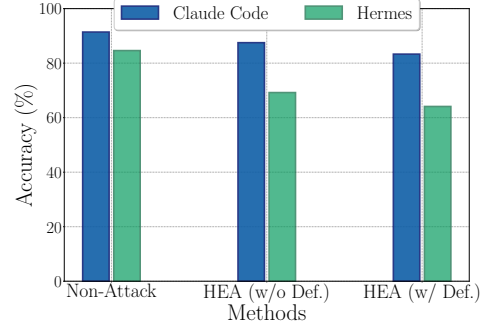


Figure 4: Task accuracy under non-attack, HEA, and defense conditions.

8 Discussion

8.1 Balancing Confidentiality and Auditability

Harness extraction also reveals a fundamental trade-off between system confidentiality and user auditability. On the one hand, AMAS providers need to protect the inference time harness. On the other hand, users need sufficient execution transparency to determine whether the system operates as expected. Such transparency is particularly important when the system can access local files or sensitive information. Completely hiding interagent communication and execution structures may reduce the risk of proprietary information leakage.

⁵<https://developers.openai.com/api/docs/models/all>

However, it may also weaken user ability to detect anomalous execution and verify system reliability (Zhao et al., 2025a). From this perspective, the harness extraction method proposed in this paper also has potential defensive applications. When internal logs are unavailable, the method can provide supplementary structural audit signals and assist users with verification. In practice, this tension can be further mitigated through an auditing mechanism that combines zero-knowledge proofs with blockchain (Narula et al., 2018; Chaliasos et al., 2025). While such a mechanism cannot eliminate information leakage entirely, it shifts auditing from the direct disclosure of execution traces to the cryptographic verification of predefined security properties, thereby preserving auditability while reducing the risk of harness extraction and subsequent distillation.

8.2 Multi-Teacher Harness Distillation

The current agent harness distillation pipeline extracts a harness from a single target system. However, an attacker with access to multiple AMAS instances, denoted by $\{\mathcal{A}_t = (\theta_t, \mathcal{H}_t)\}_{t=1}^m$, can independently extract harnesses from these systems and fuse them into a composite harness. Such multi-teacher harness distillation (Yu et al., 2025; Tian et al., 2025) can improve transfer utility while introducing additional security risks.

8.3 Agent Harness Self-Distillation

A more fundamental risk arises when the agent itself becomes the adversary. An agent with access to its own execution traces possesses knowledge of the instantiated harness h_x . If the agent operates partially outside the monitored boundary, it may construct a functionally equivalent replica $\mathcal{A}_{\text{shadow}} = (\theta, \hat{\mathcal{H}}_s)$. Here, $\hat{\mathcal{H}}_s$ is not a bitwise copy of $\mathcal{H}$, but a behaviorally indistinguishable variant. This constitutes a form of self-replication: the shadow agent retains the original system’s functional capabilities while evading the detection. Such autonomous self-distillation (Shen et al., 2025) raises concerns beyond IP protection. It may enable an agent to escape the monitored execution boundary, persist beyond its intended operational lifetime, and propagate its capabilities into uncontrolled environments.

8.4 Broader Risks of Harness Leakage

Beyond IP leakage, inference-time harness leakage may introduce broader privacy risks in AMAS.

Similar to how privacy risks in LLMs have been studied through attacks such as membership inference attacks (He et al., 2025b), the leakage of agent execution mechanisms may introduce analogous privacy threats at the system level. For example, Harness Membership Inference Attack could infer whether a specific execution pattern or trajectory fragment is associated with an agent’s inference-time harness under a given query. Such risks may reveal sensitive properties of internal workflows. Moreover, this information can provide attackers with useful priors for downstream attacks, such as targeted prompt injection (Liu et al., 2024b) or workflow manipulation (Shahroz et al., 2025; Amayuelas et al., 2024), reducing the need for blind exploration of the target agent.

9 Conclusion

In this paper, we investigate inference-time harness leakage in AMAS and formulate harness extraction as a new security problem. We propose Agent Harness Distillation (AHD), a two-stage black-box framework for extracting harness behaviors from target agents and transferring their orchestration capabilities. Extensive experiments across real-world AMAS with multiple backbone LLMs show that AHD can effectively extract transferable harness capabilities, exposing substantial IP leakage risks. We further present a deception-based defense that mitigates harness extraction while maintaining agent utility. Our work reveals a previously underexplored attack surface in AMAS. Beyond this security implication, our study also provides insights into harness self-evolution by showing the potential of extracting and transferring inference-time harness behaviors.

10 Ethical Considerations

This paper introduces an attack that extracts proprietary information about harness structures in MASs. Our goal is to improve the security of LLM agents by revealing a previously unexplored attack surface and developing rigorous red-teaming methods. We also analyze the associated risks and present corresponding defense strategies. The techniques described in this paper are intended solely for scientific research. AI assistants are used to polish the writing.

References

- Alfonso Amayuelas, Xianjun Yang, Antonis Antoniadis, Wenyue Hua, Liangming Pan, and William Yang Wang. 2024. [MultiAgent collaboration attack: Investigating adversarial attacks in large language model collaborations via debate](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 6929–6948, Miami, Florida, USA. Association for Computational Linguistics.
- Hengyu An, Minxi Li, Jinghui Zhang, Naen Xu, Chunyi Zhou, Changjiang Li, Xiaogang Xu, Tianyu Du, and Shouling Ji. 2026. [ACIArena: Toward unified evaluation for agent cascading injection](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10049–10066, San Diego, California, United States. Association for Computational Linguistics.
- Art of Problem Solving. 2025. [AIME Problems and Solutions](#). Accessed: 2025-05-15.
- Daniel Ayzenshteyn, Roy Weiss, and Yisroel Mirsky. 2025. Cloak, honey, trap: Proactive defenses against {LLM} agents. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 8095–8114.
- Stefanos Chaliasos, Denis Firsov, and Benjamin Livshits. 2025. [Towards a formal foundation for blockchain zk rollups](#). In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS ’25*, page 2714–2728, New York, NY, USA. Association for Computing Machinery.
- Mingju Chen, Can Lv, Guibin Zhang, Heng Chang, and Shiji Zhou. 2026a. Harnessforge: Joint harness and policy evolution for adaptive agent systems. *arXiv preprint arXiv:2606.01779*.
- Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. 2026b. [Harnessx: A composable, adaptive, and evolvable agent harness foundry](#). Preprint, arXiv:2606.14249.
- Vincent Cheval and Itsaka Rakotonirina. 2023. [Indistinguishability beyond diff-equivalence in proverif](#). In *2023 IEEE 36th Computer Security Foundations Symposium (CSF)*, pages 184–199.
- Zeeshan Hayder, Ali Cheraghian, Lars Petersson, Mehrtash Harandi, and Richard Hartley. 2026. [DKD: Dynamic trade-off optimization for effective knowledge distillation](#). In *The Fourteenth International Conference on Learning Representations*.
- Changyi He, Yifu Ding, Jinyang Guo, Ruihao Gong, Haotong Qin, and Xianglong Liu. 2025a. [DA-KD: Difficulty-aware knowledge distillation for efficient large language models](#). In *Forty-second International Conference on Machine Learning*.
- Yu He, Boheng Li, Liu Liu, Zhongjie Ba, Wei Dong, Yiming Li, Zhan Qin, Kui Ren, and Chun Chen. 2025b. Towards label-only membership inference attack against pre-trained large language models. In *Proceedings of the 34th USENIX Conference on Security Symposium, SEC ’25*, USA. USENIX Association.
- Yue Huang, Wenjie Wang, Han Bao, Yuchen Ma, Xiaonan Luo, Yi Nian, Haomin Zhuang, Zheyuan Liu, Yue Zhao, and Xiangliang Zhang. 2026. Memoharness: Agent harnesses that learn from experience. *arXiv preprint arXiv:2607.14159*.
- Che Jiang, Jincheng Zhong, Yu Fu, Kai Tian, Junlin Yang, Kaikai Zhao, Yuchong Wang, Tianwei Luo, Weizhi Wang, Yuxin Zuo, and 1 others. 2026. Self-improving agents in the era of experience: A survey of self-to meta-evolution.
- Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. 2026. Recursive harness self-improvement. *arXiv preprint arXiv:2607.15524*.
- Fanxiao Li, Jiaying Wu, Tingchao Fu, Natasha Jaques, Wei Zhou, and Min-Yen Kan. 2026. Flowsteer: Prompt-only workflow steering exposes planning-time vulnerabilities in multi-agent llm systems. *arXiv preprint arXiv:2605.11514*.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, and 180 others. 2024a. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.
- Songyang Liu, Chaozhuo Li, Chenxu Wang, Jinyu Hou, Zejian Chen, Litian Zhang, Zheng Liu, Qiwei Ye, Yiming Hei, Xi Zhang, and Zhongyuan Wang. 2026. Clawkeeper: Comprehensive safety protection for openclaw agents through skills, plugins, and watchers. *arXiv preprint arXiv:2603.24414*.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024b. [Formalizing and benchmarking prompt injection attacks and defenses](#). In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1831–1847, Philadelphia, PA. USENIX Association.
- Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. [Gaia: a benchmark for general ai assistants](#). In *International Conference on Learning Representations*, volume 2024, pages 9025–9049.
- Neha Narula, Willy Vasquez, and Madars Virza. 2018. [zkLedger: Privacy-Preserving auditing for distributed ledgers](#). In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 65–80, Renton, WA. USENIX Association.

- Carolina Ortega Pérez and Alaa Daffalla. 2025. [Encrypted access logging for online accounts: Device attributions without device tracking](#). In *34th USENIX Security Symposium (USENIX Security 25)*, pages 6679–6697, Seattle, WA. USENIX Association.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. [ChatDev: Communicative agents for software development](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15174–15186, Bangkok, Thailand. Association for Computational Linguistics.
- Qwen Team. 2026. [Qwen3.6-35B-A3B: Agentic coding power, now open to all](#).
- Rana Shahroz, Zhen Tan, Sukwon Yun, Charles Fleming, and Tianlong Chen. 2025. [Agents under siege: Breaking pragmatic multi-agent LLM systems with optimized prompt attacks](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9661–9674, Vienna, Austria. Association for Computational Linguistics.
- Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. 2025. [CODI: Compressing chain-of-thought into continuous space via self-distillation](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 677–693, Suzhou, China. Association for Computational Linguistics.
- Safal Shrestha, Minwu Kim, and Keith Ross. 2025. Mathematical reasoning in large language models: Assessing logical and arithmetic errors across wide numerical ranges. *arXiv preprint arXiv:2502.08680*.
- Reza Soosahabi and Vivek Namsani. 2026. Analyzing defensive misdirection against model-guided automated attacks on agentic ai systems. *arXiv preprint arXiv:2606.20470*.
- Yijun Tian, Yikun Han, Xiushi Chen, Wei Wang, and Nitesh V. Chawla. 2025. [Beyond answers: Transferring reasoning capabilities to smaller llms using multi-teacher knowledge distillation](#). In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining, WSDM '25*, page 251–260, New York, NY, USA. Association for Computing Machinery.
- Liwen Wang, Wenxuan Wang, Shuai Wang, Zongjie Li, Zhenlan Ji, Zongyi Lyu, Daoyuan Wu, and Shing-Chi Cheung. 2026a. Masleak: Investigating and exposing intellectual property leakage vulnerabilities in multi-agent systems. In *USENIX Security Symposium (USENIX Security)*.
- Ruhan Wang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Yue Yu, Junyao Yang, Kishan Panaganti, Haitao Mi, Dongruo Zhou, and Leoweiliang. 2026b. [Harness handbook: Making evolving agent harnesses readable, navigable, and editable](#). *Preprint, arXiv:2607.13285*.
- Xiaoqing Wang, Keman Huang, Bin Liang, Hongyu Li, and Xiaoyong Du. 2026c. Shadows in the code: Exploring the risks and defenses of llm-based multi-agent software development systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 37970–37978.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, and 1 others. 2024. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290.
- Yongxuan Wu, Xixun Lin, He Zhang, Nan Sun, Kun Wang, Chuan Zhou, Shirui Pan, and Yanan Cao. 2026. [CIA: Inferring the communication topology from LLM-based multi-agent systems](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17917–17933, San Diego, California, United States. Association for Computational Linguistics.
- Markus Wulfmeier, Michael Bloesch, Nino Vieillard, Arun Ahuja, Jörg Bornschein, Sandy Huang, Artem Sokolov, Matt Barnes, Guillaume Desjardins, Alex Bewley, Sarah Maria Elisabeth Bechtle, Jost Tobias Springenberg, Nikola Momchev, Olivier Bachem, Matthieu Geist, and Martin Riedmiller. 2024. [Imitating language via scalable inverse reinforcement learning](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 90714–90735. Curran Associates, Inc.
- Tianshi Xu, Huifeng Wen, and Meng Li. 2026. Adapting the interface, not the model: Runtime harness adaptation for deterministic llm agents. *arXiv preprint arXiv:2605.22166*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Zeqin Yu, Jiangqun Ni, Jian Zhang, Haoyi Deng, and Yuzhen Lin. 2025. Reinforced multi-teacher knowledge distillation for efficient general image forgery detection and localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 995–1003.
- Rui Zhao, Muhammad Shoaib, Viet Tung Hoang, and Wajih Ul Hassan. 2025a. [Rethinking tamper-evident logging: A high-performance, co-designed auditing system](#). In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS '25*, page 2624–2638, New York, NY, USA. Association for Computing Machinery.

- Zhengyang Zhao, Lu Ma, and Wentao Zhang. 2026. Training with harnesses: On-policy harness self-distillation for complex reasoning. *arXiv preprint arXiv:2605.08741*.
- Zhengyue Zhao, Xiaogeng Liu, Somesh Jha, Patrick McDaniel, Bo Li, and Chaowei Xiao. 2025b. [Can watermarks be used to detect LLM IP infringement for free?](#) In *The Thirteenth International Conference on Learning Representations*.
- Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Serkan O Arik. 2026. [Multi-agent design: Optimizing agents with better prompts and topologies](#). In *The Fourteenth International Conference on Learning Representations*.